ANGULAR BASICS: PRIMEROS PASOS

Qué es Angular, TypeScript, entorno de trabajo, Angular CLI, componentes y ciclo de vida

Apuntes revisados y ampliados
Documento elaborado a partir del fastbook original de Rodrigo Zavala

Contenido

Introducción: el proyecto Tea Minder . . . 3

1. ¿Qué es Angular? . . . 3

1.1. Historia de Angular . . . 3

2. TypeScript . . . 4

3. Configurando el workspace . . . 5

3.1. Node.js . . . 5

3.2. npm (Node Package Manager) . . . 5

3.3. Terminal . . . 6

4. Angular CLI . . . 6

4.1. Comandos más utilizados . . . 6

4.2. Instalación y primer proyecto . . . 7

4.3. Elementos que genera ng generate . . . 7

4.4. Inyección de dependencias . . . 8

5. Angular Component . . . 9

5.1. Two-way binding . . . 9

5.2. Comunicación entre componentes . . . 10

6. Ciclo de vida del componente . . . 12

7. Qué hemos aprendido . . . 13

Introducción: el proyecto Tea Minder

Este primer fastbook recorre los conceptos básicos de Angular: desde preparar la máquina e iniciar un proyecto hasta los elementos fundamentales con los que se construye una aplicación. A lo largo de toda la asignatura se trabaja sobre **un mismo ejemplo** que irá evolucionando hasta convertirse en una web app real y totalmente funcional: **Tea Minder**.

Tea Minder tendrá un sistema completo de **autenticación** (login, registro, etc.) y un **CRUD** —*Create*, *Read*, *Update*, *Delete*— para gestionar tés.

1. ¿Qué es Angular?

Angular es un **framework de desarrollo web creado por Google**. Un framework es un **conjunto de herramientas y reglas** que facilitan la construcción de aplicaciones de software: Angular proporciona una estructura sólida y coherente para desarrollar aplicaciones web de forma más rápida y organizada. Sus tres grandes ventajas son:

- **Componentes reutilizables:** en lugar de escribir todo desde cero, la aplicación se divide en piezas independientes —un componente para el encabezado, otro para el pie de página, otros para cada parte de la interfaz— que pueden reutilizarse.
- **Tareas comunes automatizadas:** Angular se encarga de actualizar la interfaz cuando cambian los datos, de gestionar la navegación entre páginas y de la comunicación con servidores, lo que ahorra tiempo y reduce errores.
- **Respaldo de Google:** es un framework robusto, muy extendido en entornos empresariales y con mejoras constantes en forma de actualizaciones.

1.1. Historia de Angular

AngularJS (2009-2016). El proyecto lo iniciaron Misko Hevery y Adam Abrons en Google en 2009, y se presentó al público en 2010. Fue uno de los primeros frameworks en popularizar el desarrollo **del lado del cliente**, donde gran parte de la lógica y la interacción se gestionan en el navegador, lo que permitió aplicaciones más dinámicas. Con el tiempo, su arquitectura generó problemas de rendimiento y mantenibilidad.

Angular 2+ (2016-actualidad). Google lanzó una **reescritura completa**, conocida como Angular 2, para resolver las limitaciones de AngularJS: arquitectura más modular, gestión más eficiente de componentes y datos, mejor rendimiento y escalabilidad. Desde entonces se publican nuevas versiones periódicamente (Angular 4, 5, 6… hasta Angular 16 en 2023).

Angular CLI (2016-actualidad). Llegó junto a Angular 2: una herramienta de línea de comandos que simplifica la configuración, creación y administración de proyectos, y la generación de componentes, módulos, servicios y otros elementos.

Angular Ivy (2019). Nuevo motor de **renderizado y compilación** que sustituyó a la tecnología anterior: mejor rendimiento, paquetes más pequeños, mejor depuración, desarrollo más rápido y librerías compartidas más eficientes.

Angular 14+ (2022-actualidad). Entre muchas mejoras destacan los **Standalone Components**: componentes totalmente independientes que no necesitan módulos. Esto cambia el paradigma y da lugar a aplicaciones **moduleless** (sin módulos).

> **★ Lo más importante**
> - Angular es un framework de Google: herramientas + reglas para construir apps web organizadas.
> - Ventajas: componentes reutilizables, tareas comunes automatizadas y respaldo empresarial de Google.
> - AngularJS (2010) y Angular 2+ (2016) son distintos: el segundo es una reescritura completa.
> - Hitos: CLI (2016), motor Ivy (2019) y Standalone Components / apps sin módulos (Angular 14+).

2. TypeScript

Hablar de Angular obliga a hablar de TypeScript, porque es su **lenguaje base desde la versión 2**. TypeScript viene a resolver varios inconvenientes de JavaScript. Sus puntos más importantes son:

- **Tipado estático:** se indica qué tipo de dato contendrá cada variable (número, cadena, booleano...). Los errores de tipo se detectan **en tiempo de compilación** en lugar de en ejecución, y el código es más seguro.
- **Superset de JavaScript:** cualquier código JavaScript válido es también TypeScript válido, así que puede introducirse en proyectos existentes sin cambios drásticos.
- **Interfaces y tipos personalizados:** las **interfaces** definen la forma que debe tener un objeto; los **types** son más flexibles. Ambos permiten estructuras de datos complejas y contratos claros entre partes del código.
- **Inferencia de tipos:** no siempre hay que escribir el tipo; TypeScript lo deduce cuando puede.
- **Compilación:** TypeScript no se ejecuta directamente en el navegador. El compilador **tsc** lo traduce a JavaScript para que cualquier navegador o motor JS lo entienda.
- **Soporte para ES6 y posteriores:** se pueden usar clases, módulos, funciones flecha y demás novedades de JavaScript junto con el tipado estático.
- **Ecosistema y herramientas:** gran comunidad y muy buena integración con editores como **Visual Studio Code**, lo que facilita escribir y depurar.

En definitiva, TypeScript añade tipado estático a JavaScript, lo que mejora la seguridad y el mantenimiento del código. Por ejemplo:

```
// Interfaz: define la forma de un objeto
interface Te {
  nombre: string;
  temperatura: number;
  favorito?: boolean;        // propiedad opcional
}

// Type: más flexible (uniones, alias...)
type Estado = 'pendiente' | 'listo';

const miTe: Te = { nombre: 'Sencha', temperatura: 80 };
let tazas = 3;               // inferido como number
// tazas = 'tres';           // error en compilación
```

Para profundizar se recomienda la documentación oficial: typescriptlang.org.

★ Lo más importante

- TypeScript = JavaScript + tipado estático; los errores de tipo aparecen al compilar, no al ejecutar.
- Es un superset: todo JavaScript válido es TypeScript válido.
- Interfaces (forma de un objeto) y types (más flexibles) definen contratos claros.
- Siempre se compila a JavaScript con tsc antes de ejecutarse.

3. Configurando el workspace

Antes de crear la primera aplicación se necesitan algunos programas y herramientas.

3.1. Node.js

Lo primero es instalar la última versión de **Node.js** desde su web oficial (nodejs.org/es), descargando siempre la versión **LTS** (*long term support*). El instalador guía paso a paso. Para comprobar que la instalación es correcta, se ejecuta en la consola:

```
node -v
v18.10.0
```

3.2. npm (Node Package Manager)

Es el **gestor de paquetes** de Node.js y su ecosistema: permite instalar, gestionar y compartir piezas de código reutilizable (paquetes o módulos) escritas en JavaScript, que pueden contener funciones, librerías o componentes. **Viene incluido con Node.js**, así que no hay que instalarlo aparte. Sus funciones principales:

Función	Qué permite
Instalar paquetes	Descargar paquetes del registro público con `npm install nombre-del-paquete`.
Administrar dependencias	Gestionar automáticamente los paquetes de los que depende la app y sus versiones.
Gestionar versiones	Fijar versiones exactas o rangos para mantener la consistencia y evitar incompatibilidades.
Publicar y compartir	Empaquetar una funcionalidad propia y compartirla con otros desarrolladores.
Ejecutar scripts	Definir scripts en el `package.json` para automatizar tests, builds, etc.

3.3. Terminal

Para ejecutar comandos hace falta una **terminal**. Sirve cualquiera, incluso la integrada en editores como VS Code. Como recomendación: **iTerm** en macOS (muy personalizable y con muchos plugins) y **Windows Terminal** en Windows (disponible en la Microsoft Store), que puede abrir distintas instancias como PowerShell o Git Bash.

> **★ Lo más importante**
> - Instala siempre la versión LTS de Node.js y comprueba con node -v.
> - npm viene con Node: instala paquetes, gestiona dependencias y versiones, publica y ejecuta scripts.
> - Cualquier terminal vale; recomendadas iTerm (macOS) y Windows Terminal (Windows).

4. Angular CLI

La **CLI de Angular** (*Command Line Interface*) es una potente herramienta para crear, gestionar y construir aplicaciones Angular desde la línea de comandos. Permite **generar componentes, servicios, módulos y otros archivos**, así como construir y servir la aplicación. Es imprescindible para cualquier desarrollador Angular porque simplifica muchas tareas comunes.

4.1. Comandos más utilizados

Comando	Qué hace
`ng new`	Crea un nuevo proyecto Angular en un nuevo directorio.
`ng generate`	Genera código Angular: componentes, servicios, módulos, pipes, directivas...
`ng serve`	Inicia un servidor de desarrollo local con recarga en vivo al cambiar el código.
`ng build`	Construye la app para producción: archivos estáticos listos para un servidor web.
`ng test`	Ejecuta las pruebas unitarias.
`ng e2e`	Ejecuta las pruebas de extremo a extremo (end-to-end).

Además, la CLI ofrece opciones para personalizar el comportamiento de cada comando y el código que genera. La referencia completa está en la documentación oficial (angular.io/cli).

4.2. Instalación y primer proyecto

Con el entorno preparado, se instala la CLI **a nivel global** (`-g`). Al no indicar versión, se instala la última estable. Después, `ng version` muestra la versión de la CLI, de Node, de npm y del sistema operativo.

```
npm install -g @angular/cli
ng version            # Angular CLI: 16.0.3 · Node: 18.10.0 · npm 8.19.2
```

Para crear la aplicación del curso se usa `ng new` con el nombre **tea-minder**, eligiendo **SCSS** como formato de estilos y, de momento, **sin routing**. Después, dentro de la carpeta creada, `ng serve` arranca el servidor local; una vez compilada, la app se ve en **http://localhost:4200**.

```
ng new tea-minder
? Would you like to add Angular routing? No
? Which stylesheet format would you like to use? SCSS
cd tea-minder
ng serve              # abrir http://localhost:4200
```

4.3. Elementos que genera ng generate

Cada tipo de elemento tiene su propio caso de uso y puede personalizarse. Por defecto se crean en `src/app/` con su archivo TypeScript asociado.

Services (`ng generate service my-service`). Un servicio es una forma de **compartir datos o funcionalidad entre distintos componentes**. Se marca con `@Injectable()`. En el ejemplo, el servicio guarda un `Subject` de RxJS, lo expone como Observable (`sharedData$`) y ofrece un método para emitir nuevos datos:

```
import { Injectable } from '@angular/core';
import { Subject } from 'rxjs';

@Injectable()
export class DataService {
  private sharedData = new Subject<string>();
  sharedData$ = this.sharedData.asObservable();

  sendData(data: string) {
    this.sharedData.next(data);
  }
}
```

Pipes (`ng generate pipe my-pipe`). Un pipe sirve para **transformar los datos antes de mostrarlos en la vista**. Implementa `PipeTransform` y su método `transform`; se usa en la plantilla por su nombre, por ejemplo `{{ texto | myPipe }}`.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'myPipe' })
export class MyPipePipe implements PipeTransform {
  transform(value: string): string {
    return value.toUpperCase();
  }
}
```

Directives (`ng generate directive my-directive`). Una directiva permite **añadir comportamiento a un elemento HTML existente** o crear un elemento HTML reutilizable. En el ejemplo, cualquier elemento con el atributo `appMyDirective` recibe fondo amarillo a través de `ElementRef`:

```
import { Directive, ElementRef } from '@angular/core';

@Directive({ selector: '[appMyDirective]' })
export class MyDirectiveDirective {
  constructor(private el: ElementRef) {
    el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

4.4. Inyección de dependencias

La **inyección de dependencias** es un concepto clave: permite **proporcionar a los componentes las instancias de las clases que necesitan**. Imagina una app que muestra una lista de productos obtenidos de una API: la inyección de dependencias conecta el componente con el servicio para que el componente pueda usar sus datos.

```
import { Component } from '@angular/core';
import { ProductService } from './product.service';

@Component({
  selector: 'app-product',
  template: '<ul><li *ngFor="let product of products">{{ product }}</li></ul>'
})
export class ProductComponent {
  products: string[] = [];

  constructor(private productService: ProductService) {
    this.products = this.productService.getProducts();
  }
}
```

Analizando el ejemplo:

- Existe un servicio `ProductService` con un método `getProducts()` que devuelve la lista de productos.
- `ProductComponent` recibe una instancia de `ProductService` **en su constructor**: Angular la crea y la inyecta, el componente no hace new.

- En el constructor se llama a `getProducts()`, se guarda el resultado en `products` y la plantilla lo recorre con `*ngFor` para mostrarlo en una lista.

La inyección de dependencias mantiene el código organizado y favorece la reutilización y el mantenimiento: conecta componentes y servicios de forma fluida y coherente.

> **★ Lo más importante**
> - Imprescindibles: ng new, ng generate, ng serve (localhost:4200), ng build, ng test, ng e2e.
> - La CLI se instala global con npm install -g @angular/cli; ng version comprueba la instalación.
> - Tea Minder se crea con ng new tea-minder, estilos SCSS y sin routing (de momento).
> - Servicio = compartir datos/lógica · Pipe = transformar datos en la vista · Directiva = comportamiento en un elemento HTML.
> - Inyección de dependencias: Angular entrega el servicio al componente a través del constructor.

5. Angular Component

Un componente es el **bloque de construcción básico** de una aplicación Angular. Consiste en una **clase TypeScript** y una **plantilla HTML** (más sus estilos). Se crea con `ng generate component my-component`, que genera en `src/app/` los archivos HTML, SCSS y TypeScript. El decorador `@Component` define:

- `selector`: la etiqueta HTML con la que se usa el componente (p. ej. `<app-my-component>`).
- `templateUrl`: la ruta a su plantilla HTML (o `template` para escribirla en línea).
- `styleUrls`: la ruta (o rutas) a sus hojas de estilo.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-my-component',
  templateUrl: './my-component.component.html',
  styleUrls: ['./my-component.component.css']
})
export class MyComponentComponent {
  title = 'my-component';
}
```

5.1. Two-way binding

El **two-way binding** o doble enlace mantiene **sincronizados automáticamente** los datos de la interfaz (HTML) y del componente (TypeScript): cuando cambian en la plantilla, se actualizan en el componente, y viceversa. Por ejemplo, lo que se escribe en un cuadro de texto se refleja al instante en una variable del componente; y si esa variable cambia en el código, el cuadro se actualiza solo.

Se implementa con la directiva **[(ngModel)]**, que **combina en un solo paso el enlace de la propiedad y el evento de cambio**:

```
// app.component.ts
export class AppComponent {
  mensaje: string = '';   // variable para almacenar el mensaje
}

<!-- app.component.html -->
<input [(ngModel)]="mensaje" placeholder="Escribe un mensaje">
<p>El mensaje es: {{ mensaje }}</p>
```

Nota práctica (no aparece en el fastbook): para usar ngModel hay que importar FormsModule, en el módulo o en el array imports del componente si es standalone.

5.2. Comunicación entre componentes

Las aplicaciones Angular se basan en la **modularización y reutilización** de componentes. Por eso cada componente debe tener **una única responsabilidad**: si uno es muy complejo, lo ideal es dividirlo en componentes más pequeños y mantenibles, que además podrán reutilizarse. Como consecuencia, hay que saber cómo pasar información entre ellos según la relación que exista:

1. De hijo a padre. El hijo **emite un evento** con `@Output()` y un `EventEmitter`, y el padre lo escucha en la plantilla. Es como si el hijo levantara la mano y el padre respondiera. El dato emitido llega al padre en `$event`.

```
// child.component.ts
@Component({
  selector: 'app-child',
  template: '<button (click)="emitData()">Enviar datos al padre</button>'
})
export class ChildComponent {
  @Output() dataEmitted = new EventEmitter<string>();
  emitData() { this.dataEmitted.emit('¡Hola desde el hijo!'); }
}

// parent.component.ts
@Component({
  selector: 'app-parent',
  template: `<app-child (dataEmitted)="handleData($event)"></app-child>
             <p>{{ receivedData }}</p>`
})
export class ParentComponent {
  receivedData: string = '';
  handleData(data: string) { this.receivedData = data; }
}
```

2. De padre a hijo. El padre pasa datos a través de **propiedades de entrada** (*input properties*) marcadas con `@Input()` en el hijo. Es como si el padre le diera algo al hijo antes de que empiece a funcionar, y el hijo lo usa para mostrar información o actuar.

```
@Component({ selector: 'app-child', template: '<p>{{ message }}</p>' })
export class ChildComponent {
  @Input() message = '';
}

<!-- en la plantilla del padre -->
<app-child [message]="'Hola desde el padre'"></app-child>
```

3. Entre hermanos (mediante un servicio). Cuando dos componentes están al mismo nivel, se usa un **servicio compartido** como mensajero: uno coloca datos en él (el `DataService` visto en 4.3, con `sendData()`) y el otro se **suscribe** para recibirlos, sin relación directa entre ellos.

```
// sibling2.component.ts
export class Sibling2Component {
  receivedData: string = '';

  constructor(private dataService: DataService) {
    this.dataService.sharedData$.subscribe(data => {
      this.receivedData = data;
    });
  }
}
```

4. Totalmente independientes. Cuando los componentes necesitan comunicarse pero no están en la misma jerarquía, se recurre a **servicios de eventos o librerías externas**, que permiten emitir y escuchar eventos en cualquier parte de la aplicación, sin importar dónde estén los componentes.

Relación	Mecanismo
Padre → hijo	`@Input()` en el hijo + property binding `[prop]` en el padre
Hijo → padre	`@Output()` + `EventEmitter` en el hijo + event binding `(evento)` en el padre
Hermanos	Servicio compartido (Subject / Observable) al que ambos acceden
Independientes	Servicios de eventos o librerías externas

★ Lo más importante

- Componente = clase TypeScript + plantilla HTML + estilos, definido con @Component (selector, templateUrl, styleUrls).
- [(ngModel)] sincroniza vista y componente en ambos sentidos (propiedad + evento en un paso).
- Un componente, una responsabilidad: si crece demasiado, se divide.
- Padre → hijo: @Input · Hijo → padre: @Output + EventEmitter · Hermanos: servicio · Independientes: eventos/librerías.

6. Ciclo de vida del componente

El ciclo de vida de un componente consta de varios eventos que ocurren **desde su creación hasta su destrucción**. Estos eventos son **puntos de gancho** (*hooks*) que permiten ejecutar código en momentos concretos, útiles para inicializar, actualizar y limpiar. Se ejecutan en este orden:

Hook	Cuándo se ejecuta y para qué sirve
1. `ngOnChanges`	Cuando cambian los datos de entrada (@Input) del componente.
2. `ngOnInit`	Después de que Angular inicialice el componente. Buen lugar para inicializaciones adicionales.
3. `ngDoCheck`	Cada vez que Angular realiza la detección de cambios. Permite comprobaciones personalizadas.
4. `ngAfterContentInit`	Después de proyectar el contenido (externo) en el componente.
5. `ngAfterContentChecked`	Después de verificar el contenido proyectado.
6. `ngAfterViewInit`	Después de inicializar la vista del componente.
7. `ngAfterViewChecked`	Después de verificar la vista del componente.
8. `ngOnDestroy`	Justo antes de destruir el componente. Para limpiar: cancelar suscripciones y liberar recursos.

Para usar un hook, la clase implementa su interfaz (por ejemplo `OnInit`, `OnDestroy`) y define el método correspondiente:

```
export class TeaListComponent implements OnInit, OnDestroy {
  private sub?: Subscription;

  constructor(private dataService: DataService) {}

  ngOnInit() {
    this.sub = this.dataService.sharedData$.subscribe(d => console.log(d));
  }

  ngOnDestroy() {
    this.sub?.unsubscribe();   // evita fugas de memoria
  }
}
```

★ Lo más importante

- Ocho hooks, en orden: OnChanges, OnInit, DoCheck, AfterContentInit, AfterContentChecked, AfterViewInit, AfterViewChecked, OnDestroy.
- ngOnChanges solo reacciona a cambios en los @Input.
- ngOnInit: inicializaciones · ngOnDestroy: limpieza (suscripciones, recursos).
- Los hooks Content tratan el contenido proyectado; los hooks View, la vista propia del componente.

7. Qué hemos aprendido

En este primer bloque hemos visto qué es Angular y cómo ha evolucionado desde AngularJS hasta los Standalone Components y las aplicaciones sin módulos; por qué TypeScript y su tipado estático son la base del framework; cómo preparar el entorno con Node.js (LTS), npm y una terminal; cómo instalar y usar la Angular CLI para crear Tea Minder, servirla en local y generar código; y los pilares de cualquier aplicación: servicios, pipes, directivas, inyección de dependencias, componentes, two-way binding, comunicación entre componentes y ciclo de vida.

Estos conceptos y comandos son imprescindibles para dar los primeros pasos en Angular, y se irán aplicando sobre Tea Minder en los siguientes temas hasta convertirla en una web app completa con autenticación y CRUD.

¡Enhorabuena! Documento completado